Technical Report: Fault Tolerance in Loan Application Systems

1. Introduction

In a financial system like a Loan Application module, data integrity and availability are paramount. A database crash during an application process can lead to data loss, inconsistent states (e.g., loan created but documents missing), and poor user experience. This report outlines strategies to achieve fault tolerance.

---

2. Core Strategies

A. Database Transaction Integrity (ACID)

The most fundamental layer is ensuring **Atomicity**. Using Spring's `@Transactional` annotation ensures that if any part of the loan submission fails (DB crash), the entire operation is rolled back.

```
@Transactional
public LoanApplication submitApplication(LoanApplicationDTO dto) {
    // 1. Save Application
    // 2. Save Documents
    // 3. Update User Status
    // If DB crashes here, everything above is rolled back.
}
```

B. Resilience4j: Retry & Circuit Breaker

Retry allows the application to automatically re-attempt a database operation if it fails due to a transient network issue or a temporary DB restart.

Circuit Breaker detects if the database is consistently failing and "trips," preventing the application from sending more requests and allowing the DB to recover.

C. Message Queuing (Asynchronous Persistence)

By introducing a Message Broker (RabbitMQ/Kafka), we decouple the user request from the database write.

1. User submits loan.
2. Application puts the loan data into a **Queue**.
3. Application returns "Application Received" to the user.
4. A **Consumer** process reads from the queue and writes to the DB. *If the DB is down, the message stays safely in the queue until the DB is back.*

---

3. Database-Level Resilience (PostgreSQL)

High Availability (HA) Clusters

Deploying PostgreSQL in a **Primary-Replica** configuration with a tool like **Patroni** or **repmgr** ensures that if the Primary node fails, a Replica is automatically promoted to Primary.

Write-Ahead Logging (WAL)

PostgreSQL uses WAL to ensure that even if the server crashes, it can replay the logs upon restart to restore the data to its last consistent state.

---

4. Summary Table

Strategy	Failure Handled	Complexity	Impact
Transactions	Data Inconsistency	Low	High
Retries	Transient Downtime	Medium	Medium
Message Queues	Extended Downtime	High	Very High
DB Clusters	Hardware/Service Failure	High	Critical

---

5. Conclusion

A combination of **Spring Transaction Management** for data integrity and **Message Queues** for availability provides the most robust fault-tolerant architecture for a loan application system.